

System Project Docs: (2) Clone and edit your policy

2. Clone and Edit Your Policy

Work on your policy from your own computer

Your team writes one file: the policy.

Everything else on the car is provided for you.

In this step, you will clone the policy repository onto **your own computer** and edit your team's policy there.

Do not develop directly on the car.

Edit your code on your own computer. Later, you will copy the finished policy to the car, build it, and run it there.

Before you start

You need two things on your own computer:

1. **Git**, to download the repository.
2. A **terminal**: Terminal on macOS or PowerShell on Windows.

Check that Git is installed

Open your terminal and run:

```
git --version
```

If Git is installed, the command will print its version number.

If the command is not recognised:

macOS

Run:

```
xcode-select --install
```

This installs Git together with Apple's command-line tools.

Windows

Install [Git for Windows](https://git-scm.com/download/win)  (<https://git-scm.com/download/win>), or run the following command in PowerShell:

```
winget install --id Git.Git
```

Both methods add `git` to your PATH so that you can use it from PowerShell.

No additional SSH setup is required.

`ssh` and `scp`, which you will use later, are already included with modern versions of macOS and Windows.

1. Get the code

Clone the policy repository onto **your own computer**.

You do **not** need to download the rest of the car software or install ROS locally.

```
git clone https://gitlab.unimelb.edu.au/dream/ai4r_policy.git
cd ai4r_policy
```

After running these commands, you should be inside the `ai4r_policy` folder on your own computer.

2. What is in the repository

```
ai4r_policy/
├── scripts/
│   └── policy_node.py    <-- the file your team edits
├── config/
│   ├── ai4r_policy.yaml          <-- your trigger + required sensors
│   ├── traxxas_vehicle_interface.yaml
│   ├── oakd_cone_detector.yaml
│   └── bno08x_imu_interface.yaml
└── README.md
```

You are expected to change:

- `scripts/policy_node.py` — your team's policy.
- Your own settings in `config/ai4r_policy.yaml`.

Do not modify the other car nodes.

The vehicle, camera, lidar and IMU software is maintained for you. You do not need to change its code.

3. Where to write your code

Open:

```
scripts/policy_node.py
```

Find the method:

```
calculate_policy_actions()
```

Inside it, you will find a clearly marked section:

```
# =====  
# START OF: INSERT POLICY CODE BELOW HERE  
# =====  
...  
# =====  
# END OF: INSERT POLICY CODE ABOVE HERE  
# =====
```

Write your policy between these two markers.

The surrounding framework code is already provided and should generally be left alone.

`calculate_policy_actions()` runs once per policy step. Inside your policy block, you assign:

- `drive_action`
- `steering_action`
- `camera_pan_action` (*optional*)
- `debug1` and `debug2` (*optional*)

The code below the marked section returns these values for you. You do **not** publish ROS messages or manage the vehicle state machine yourself. The surrounding framework handles those tasks.

Coming from the Dream Gym notebook?

The notebook and the physical car do not use exactly the same observation names, cone colour numbering, or units.

See [Notebook → Policy Node Translation Guide](#)

(<https://canvas.lms.unimelb.edu.au/courses/238681/pages/system-project-docs-2a-pass-your-notebook-policy-to-the-car>) for a side-by-side translation table, common translation traps, and a worked example.

What you are given: observations

Unless stated otherwise, distances are in **metres** and angles are in **radians**.

Car-frame quantities use `base_link`: **+x forward, +y left, +z up**.

Cones

Parallel lists: `x_coords`, `y_coords`, `z_coords`, `cone_colour`, and `cone_confidence`.
Index `i` describes one cone.

Use: `ConeDetection.COLOR_YELLOW` and `ConeDetection.COLOR_BLUE` (values `1` and `2`).

A valid empty detection frame gives `num_cones == 0` while sensor availability remains `True`.

Wheel speed

Unsigned metres per second. It does **not** indicate whether the car is travelling forward or in reverse.

`None` means no valid measurement — it is **not** a measured speed of zero.

Lidar

`lidar_ranges[i]` corresponds to:

```
angle_min + i * angle_increment
```

Angles are in the lidar's own frame and are not rotated into the car body frame. `inf` can mean no return, while `NaN` is not a zero-distance obstacle.

IMU

Provides the orientation quaternion and derived roll, pitch and heading. Heading is re-zeroed relative to the moment the policy starts. Specific force **includes gravity**.

Timing

`dt` is the real elapsed time between policy steps. On the first step:

```
dt == 0.0
```

Guard against dividing by `dt`.

The actions you set

Actions must be finite, normalized values in the range `[-1, 1]`.

Action	Meaning
<code>drive_action</code>	Motor effort. This is not a target speed in metres per second.
<code>steering_action</code>	Steering command. <code>0</code> means straight ahead.
<code>camera_pan_action</code>	Independent camera-pan target. <code>None</code> holds the current position.
<code>debug1</code> , <code>debug2</code>	Your own numeric debugging values, which you can monitor in Foxglove.

Values outside `[-1, 1]` are clipped. If your code produces an invalid value or raises an error, the policy stops rather than sending an unsafe command to the car.

Never sleep, loop, or wait inside your policy code.

The policy runs on a single thread. Blocking your policy also delays the watchdog that helps keep the car safe.

Where this is documented

The source of truth is the code you are editing.

Read the large comment block immediately above the `INSERT POLICY CODE` markers in `scripts/policy_node.py`, together with the comments in the four `config/` files.

These comments are maintained with the code. Your own clone is the exact version used by the car, so this is the version you should trust.

This Canvas page is a summary. For deeper technical details, refer to the upstream documentation:

What you want	Where
Policy ROS topics, message types, directions and rates	<u>ai4r_policy README</u> (https://gitlab.unimelb.edu.au/dream/ai4r_policy/-/blob/main/README.md)
Exact message fields, units and cone colour constants	<u>dream_interfaces messages</u> (https://gitlab.unimelb.edu.au/dream/dream_interfaces/-/tree/main/msg)
How cones are detected, filtered and framed	<u>oakd_cone_detector README</u> (https://gitlab.unimelb.edu.au/dream/oakd_cone_detector/-/blob/main/README.md)
Vehicle states and the Enable / Disable / Disarm contract	<u>Vehicle ROS interface</u> (https://gitlab.unimelb.edu.au/dream/traxxas_vehicle_interface/-/blob/main/docs/interface.md)
Namespace, coordinate frames and the policy /	<u>DREAM AI4R runtime guide</u> (https://gitlab.unimelb.edu.au/dream/dream_system/-/blob/main/docs/ai4r-runtime.md)

What you want	Where
vehicle control split	

Accessing GitLab from off campus?

The DREAM repositories are hosted on the University's GitLab. Use the same **GlobalProtect VPN** session that you use to connect to the car.

4. Choosing when your policy runs

Open:

```
config/ai4r_policy.yaml
```

Here you choose:

- what triggers one execution of your policy, and
- which sensors must be present and fresh before your policy can run.

```
policy_update_mode: cone_detection # cone_detection | lidar | timer
required_sensors: [cone_detections] # what must be healthy for your policy to run
```

If a required sensor is missing or stale:

the policy stops and publishes zero commands. It will not resume automatically; you must explicitly ask it to run again.

The controls for starting and stopping the policy are covered in [Run the Car](https://canvas.lms.unimelb.edu.au/courses/238681/pages/system-project-docs-4-run-the-car) (<https://canvas.lms.unimelb.edu.au/courses/238681/pages/system-project-docs-4-run-the-car>).

5. Working as a team

For your **first session**, keep things simple:

1. Clone the repository onto your own computer.
2. Edit your policy locally.
3. Copy the file to the car when you are ready to test it.

You do not need to establish a team Git workflow immediately.

Once multiple people begin editing the policy:

move to a proper Git workflow with a team fork and branches.

See [Team Git Workflow](#)

(<https://canvas.lms.unimelb.edu.au/courses/238681/pages/system-project-docs-2b-working-as-a-team-with-git>) for the recommended approach to forking `ai4r_policy`, developing different ideas on branches, and keeping your fork up to date with framework changes.

6. What happens next

Editing the repository on your own computer does **not** automatically update the car.

Next step → Transfer and build

Copy your work onto the car and build it by following [3. Transfer and Build](#) (<https://canvas.lms.unimelb.edu.au/courses/238681/pages/system-project-docs-3-transfer-and-build>).